



Project Documentation

ITI Summer Internship 2026
Flutter Graduation Project

Prepared by: Roaa Jouda

1. Project Overview

Reel is a cross-platform mobile application built with Flutter that allows users to discover, search, and organise movies into personalised lists. The app connects to The Movie Database (TMDB) API to provide real-time, up-to-date movie data, and uses Firebase Authentication to manage secure user accounts. Movie lists are stored locally on the device using Hive, a fast NoSQL database, ensuring data persists across app sessions per individual user.

The application was built as a graduation project for the ITI Summer Internship 2026 Flutter track, demonstrating a complete, production-ready mobile app architecture.

2. Features

Feature	Description
Authentication	Secure sign-up and login using Firebase (email & password)
Home Screen	Curated sections: Now Playing, Most Popular, Top Rated, Upcoming
Genre Browsing	Filter movies by genre using horizontal chip navigation
Movie Details	Full detail view: overview, rating, release year, similar movies
Search	Live search across the entire TMDB catalogue with paginated results
Favourites	Save and manage a personal favourites list
My Lists	Three personal lists: Watching, Want to Watch, Watched
Profile	View name, email, and stats (favourites, watched, want-to-watch count)
Persistent Storage	Movie lists survive app restarts, stored locally per user account
Pagination	Infinite scroll on all movie list screens

3. Technologies Used

Category	Technology	Version / Notes
Framework	Flutter / Dart	SDK ^3.13.0
State Management	Provider / ChangeNotifier	^6.1.5
Authentication	Firebase Authentication	^6.6.1
Movie Data	TMDB REST API v3	Bearer token auth
Local DB	Hive + hive_flutter	Per-user typed boxes
Auth State	SharedPreferences	^2.5.5
Networking	http	^1.6.0
Pagination	infinite_scroll_pagination	^4.1.0
Firebase Core	firebase_core	^4.14.0

4. Architecture Explanation

The project follows a feature-first MVC architecture. Each feature is self-contained with its own View, Provider, Controller, and Service layers. This enforces a strict separation of concerns and makes the codebase easy to scale and maintain.

4.1 Folder Structure

- `lib/core/constants.dart` — TMDB token (gitignored)
- `lib/core/exceptions/` — Custom exception classes (AuthException, ApiException, etc.)
- `lib/core/services/api_service.dart` — All TMDB HTTP requests
- `lib/core/services/firebase_auth_service.dart` — Firebase sign-in/sign-up/logout
- `lib/core/services/hive_service.dart` — Per-user Hive boxes for movie lists
- `lib/core/services/local_storage_service.dart` — SharedPreferences for auth state
- `lib/core/models/movie_record.dart` — Hive-annotated local model
- `lib/core/models/movies.dart` — TMDB list response model (Movies, Result, Genre)
- `lib/core/models/single_movie.dart` — TMDB single movie detail model
- `lib/features/auth/` — Login, Sign Up (controller, provider, view)
- `lib/features/home/` — Home screen with movie sections
- `lib/features/category/` — Genre filter + genre movie list
- `lib/features/movie_list/` — Paginated "see all" screens
- `lib/features/movie-details/` — Full detail + similar movies
- `lib/features/search/` — Live search
- `lib/features/favourites/` — Favourites list
- `lib/features/my_lists/` — Watching / Want to Watch / Watched
- `lib/features/profile/` — Profile, stats, logout
- `lib/widgets/` — Shared widgets (MovieCard, MovieSection, NowPlayingBanner)

4.2 Data Flow

Every user action flows through four clearly defined layers. The View never talks directly to a Service; all business logic is owned by the Controller.

Layer	Responsibility	Example
View (Screen)	Renders UI, dispatches user events	LoginScreen calls provider.login()
Provider	Holds state, calls controller, notifies listeners	AuthProvider updates AuthState
Controller	Orchestrates services, applies business rules	AuthController validates inputs
Service	Talks to external systems (API, DB, Firebase)	ApiService fetches TMDB endpoint

5. Installation Instructions

Prerequisites

- Flutter SDK ≥ 3.13 (with Dart ≥ 3.0)

- Android Studio or VS Code with the Flutter & Dart extensions
- A physical device or emulator (Android / iOS)
- A Firebase project (see Section 7)
- A TMDb account (see Section 6)

Steps

Step	Command / Action
1. Clone the repo	<code>git clone https://github.com/roaajouda/iti-final-project.git</code>
2. Enter directory	<code>cd iti-final-project</code>
3. Install packages	<code>flutter pub get</code>
4. Add TMDb token	Create <code>lib/core/constants.dart</code> (see Section 6)
5. Add Firebase config	Run <code>flutterfire configure</code> (see Section 7)
6. Run the app	<code>flutter run</code>

6. API Setup — TMDb

All movie data is fetched from The Movie Database (TMDb) REST API v3 using a Bearer token (Read Access Token v4). This token is personal and must never be committed to a public repository.

The file `lib/core/constants.dart` is **gitignored**. You must create it manually.

Steps to get your TMDb token

- Create a free account at <https://www.themoviedb.org>
- Go to Settings → API
- Copy your API Read Access Token (v4 auth)
- Create the file `lib/core/constants.dart` with the content below

`lib/core/constants.dart`

```
class Constants {
  static const String tmdbToken = 'YOUR_TMDB_READ_ACCESS_TOKEN_HERE';
}
```

The `ApiService` automatically attaches this token as a Bearer Authorization header on every HTTP request to `api.themoviedb.org`.

7. Firebase Setup

Firebase Authentication handles all user sign-up and login operations. Firebase config files are personal and gitignored — each developer must connect their own Firebase project.

Gitignored files you must generate

File	Platform	How to get it
android/app/google-services.json	Android	flutterfire configure
ios/Runner/GoogleService-Info.plist	iOS / macOS	flutterfire configure
lib/firebase_options.dart	All platforms	flutterfire configure

Setup steps

- Go to <https://console.firebase.google.com> and create a new project
- Under Authentication → Sign-in method, enable Email/Password
- Install the Firebase CLI: `npm install -g firebase-tools`
- Install the FlutterFire CLI: `dart pub global activate flutterfire_cli`
- From the project root, run: `flutterfire configure`
- Select your project — the CLI generates all three config files automatically

8. Database Explanation

The application uses two complementary local storage mechanisms, each serving a different purpose.

9.1 SharedPreferences — Auth State

SharedPreferences stores lightweight key-value data to remember whether a user is logged in and to cache their display name locally.

Key	Type	Purpose
isLoggedIn	bool	Whether an active session exists
userName	String	Display name shown on home and profile

9.2 Hive — Movie Lists

Hive is a fast, lightweight NoSQL database that stores data in binary format on the device. It is used to persist the user's movie lists across sessions.

Per-user isolation: Each box name includes the Firebase UID (e.g. favourites_abc123), so multiple users on the same device each have completely separate data. Boxes are opened after login and closed on logout.

Box Name	Type	Contents
favourites_<uid>;	Box<MovieRecord>;	Movies saved as favourites
watch_now_<uid>;	Box<MovieRecord>;	Movies the user is currently watching
watch_later_<uid>;	Box<MovieRecord>;	Movies the user wants to watch
watched_<uid>;	Box<MovieRecord>;	Movies the user has finished watching

MovieRecord — stored model

Field	Type	Description
id	int	TMDB movie ID — used as the Hive box key

title	String	Movie title
posterPath	String?	Relative path for poster image (TMDB CDN)
releaseYear	int?	Year extracted from release date
voteAverage	double	TMDB community rating (0.0 – 10.0)

9. Application Flow

The following describes the complete user journey from app launch through to all major screens.

Step	Screen / Action	Description
1	App Launch	Hive initialises, adapter registered, Firebase initialises
2	Session Check	If user already logged in: open Hive boxes → go to Home If not logged in: go to Login Screen
3	Login / Sign Up	User enters credentials → Firebase Authentication → open Hive boxes → Home Screen
4	Home Screen	Loads Now Playing banner, genre chips, Popular / Top Rated / Upcoming sections from TMDB
5	Genre Chip Tap	Navigates to Category Screen filtered by the selected genre with infinite scroll
6	Movie Card Tap	Navigates to Movie Details Screen with full info and similar movies
7	Movie Details	User can add/remove movie from: Favourites, Watching, Want to Watch, or Watched
8	Search	Live search against TMDB with paginated results grid
9	Favourites	Displays saved favourites grid from local Hive box
10	My Lists	Tabbed screen: Watching / Want to Watch / Watched — reads from Hive
11	Profile	Shows avatar, name, email, stat counts; logout clears session and closes Hive boxes

10. Known Limitations

No offline movie browsing

All movie data is fetched live from the TMDB API. With no internet connection, Home and Search screens show an error. Only already-saved local lists remain accessible offline.

Local storage only

Hive stores movie lists on the device only. Data does not sync across multiple devices for the same account. Uninstalling the app permanently deletes the lists.

English only

The UI is in English. TMDB responses are returned in English by default. No multi-language support has been implemented.

Images require internet

Poster images are loaded from image.tmdb.org and are not cached for offline viewing.

No user reviews or ratings

Users can categorise movies into lists but cannot write their own reviews or assign a personal rating to a movie.

No push notifications

The app does not send any alerts, reminders, or update notifications.

Movie data provided by The Movie Database (TMDB) — <https://www.themoviedb.org>. This product uses the TMDB API but is not endorsed or certified by TMDB. Built with Flutter and Firebase.